

Basic Data Cleaning

Report

Introduction

This project uses **RFM analysis (Recency, Frequency, Monetary)** to segment customers based on their purchasing behaviour. The raw transaction dataset is cleaned and transformed to calculate RFM metrics for each customer. These metrics are then used to group customers into meaningful segments that help understand customer value and support business decisions such as targeted marketing and customer retention.

Attributes of the Raw Dataset

Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
---------	-----------	-------------	----------	-------------	-------	-------------	---------

The raw dataset consists of transactional-level information captured from an online retail system. Each row represents a single product purchased within an invoice. The main attributes and their roles in RFM analysis are described below:

1. Invoice

This column represents the unique invoice number for each transaction. Multiple rows can share the same invoice number because a single invoice can contain multiple products. It helps identify individual transactions and is useful when calculating purchase frequency.

2. Stock Code

This is the unique identifier for each product. It helps distinguish between different items sold by the business. While not directly used in RFM calculations, it is useful for product-level analysis and data validation.

3. Description

This column contains the name or description of the product purchased. It provides human-readable information about items but is not required for RFM analysis and may contain inconsistencies or missing values.

4. Quantity

This indicates the number of units purchased for a specific product in a transaction. It is a critical field for calculating the total monetary value of each transaction. Negative values may appear due to returns or cancellations and must be handled during cleaning.

5. Invoice Date

This represents the date and time when the transaction occurred. It is used to calculate **Recency**, which measures how recently a customer made a purchase.

The column must be converted into a proper datetime format for time-based analysis.

6. Price

This is the unit price of each product. It is used along with Quantity to compute the total transaction value. Negative or zero prices are invalid for sales analysis and must be filtered out during data cleaning.

7. Customer ID

This uniquely identifies each customer. It is essential for grouping transactions at the customer level when computing Frequency and Monetary values. Rows with missing Customer IDs cannot be used in RFM analysis.

8. Country

This indicates the customer's country. While not directly used in RFM scoring, it is useful for geographical analysis and understanding customer distribution. In this dataset, most records belong to the United Kingdom.

Raw Dataset Issues (Before Cleaning)

Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	
489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	01-12-2009 07:45	6.95	13085	United Kingdom	
489434	79323P	PINK CHERRY LIGHTS	12	01-12-2009 07:45	6.75	13085	United Kingdom	
489434	79323W	WHITE CHERRY LIGHTS	12	01-12-2009 07:45	6.75	13085	United Kingdom	
489434	22041	RECORD FRAME 7" SINGLE SIZE	48	01-12-2009 07:45	2.1	13085	United Kingdom	
489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	01-12-2009 07:45	1.25	13085	United Kingdom	
489434	22064	PINK DOUGHNUT TRINKET POT	24	01-12-2009 07:45	1.65	13085	United Kingdom	
489434	21871	SAVE THE PLANET MUG	24	01-12-2009 07:45	1.25	13085	United Kingdom	
489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	01-12-2009 07:45	5.95	13085	United Kingdom	
489435	22350	CAT BOWL	12	01-12-2009 07:46	2.55	13085	United Kingdom	
489435	22349	DOG BOWL , CHASING BALL DESIGN	12	01-12-2009 07:46	3.75	13085	United Kingdom	
489435	22195	HEART MEASURING SPOONS LARGE	24	01-12-2009 07:46	1.65	13085	United Kingdom	
489435	22353	LUNCHBOX WITH CUTLERY FAIRY CAKES	12	01-12-2009 07:46	2.55	13085	United Kingdom	
489436	48173C	DOOR MAT BLACK FLOCK	10	01-12-2009 09:06	5.95	13078	United Kingdom	
489436	21755	LOVE BUILDING BLOCK WORD	18	01-12-2009 09:06	5.45	13078	United Kingdom	
489436	21754	HOME BUILDING BLOCK WORD	3	01-12-2009 09:06	5.95	13078	United Kingdom	
489436	84879	ASSORTED COLOUR BIRD ORNAMENT	16	01-12-2009 09:06	1.69	13078	United Kingdom	
489436	22119	PEACE WOODEN BLOCK LETTERS	3	01-12-2009 09:06	6.95	13078	United Kingdom	
489436	22142	CHRISTMAS CRAFT WHITE FAIRY	12	01-12-2009 09:06	1.45	13078	United Kingdom	
489436	22296	HEART IVORY TRELLIS LARGE	12	01-12-2009 09:06	1.65	13078	United Kingdom	
489436	22295	HEART FILIGREE DOVE LARGE	12	01-12-2009 09:06	1.65	13078	United Kingdom	
489436	22109	FULL ENGLISH BREAKFAST PLATE	16	01-12-2009 09:06	3.39	13078	United Kingdom	
489436	22107	PIZZA PLATE IN BOX	4	01-12-2009 09:06	3.75	13078	United Kingdom	
489436	22194	BLACK DINER WALL CLOCK	2	01-12-2009 09:06	8.5	13078	United Kingdom	
489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	01-12-2009 09:06	4.65	13078	United Kingdom	
489436	82582	AREA PATROLLED METAL SIGN	12	01-12-2009 09:06	2.1	13078	United Kingdom	
489436	21181	PLEASE ONE PERSON METAL SIGN	12	01-12-2009 09:06	2.1	13078	United Kingdom	
489436	21756	BATH BUILDING BLOCK WORD	3	01-12-2009 09:06	5.95	13078	United Kingdom	

The raw transactional dataset cannot be used directly for RFM analysis due to several data quality problems. These issues must be addressed during the data cleaning stage:

1. Negative Quantity Values

Some transactions contain negative values in the *Quantity* column. These

usually represent product returns, cancellations, or data entry errors. Since RFM analysis focuses on actual purchases, such records can distort frequency and monetary calculations and therefore need to be removed or handled separately.

2. **Negative Price Values**

The *Price* column contains negative values in certain rows, which is not valid for sales transactions. These entries can incorrectly reduce the total monetary value of a customer and must be filtered out during preprocessing.

3. **Missing Total Amount Column**

The dataset does not include a pre-calculated *Total Amount* or *Sales Value* column. To compute the Monetary value for RFM analysis, a new column must be created by multiplying Quantity × Price for each transaction.

4. **Incorrect Data Types**

Columns such as *Invoice Date* are not stored in proper datetime format, and numerical columns like *Quantity* and *Price* may be treated as strings. These data types need to be converted to appropriate formats to enable correct calculations and time-based analysis.

5. **Null Customer ID Values**

Some records have missing *Customer ID* values. Since RFM analysis is performed at the customer level, transactions without a customer identifier cannot be assigned to any customer segment and must be removed from the dataset.

6. **Duplicate and Noisy Records**

The dataset may contain duplicate rows or inconsistent product descriptions, which can affect frequency counts and revenue calculations. These records should be reviewed and cleaned to ensure accurate analysis.

Procedure

Tools Used

The data cleaning and preprocessing steps were performed using **Jupyter** Notebook with Python libraries such as **Pandas and NumPy**, which allowed interactive exploration, transformation, and validation of the dataset before performing RFM analysis.

Code Explanation Step by Step

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib
```

Explanation

In this step, the required Python libraries are imported to support data cleaning, analysis, and visualization:

- **Pandas (pd)**
Used for loading the dataset into *DataFrames* and performing data manipulation such as filtering rows, handling missing values, grouping data by customer, and creating new columns.
- **NumPy (np)**
Provides numerical operations and support for handling missing values, mathematical calculations, and efficient array operations during data preprocessing.
- **Matplotlib (plt)**
Used for creating visualizations such as histograms and bar charts to explore data distributions and validate results during analysis.

This step sets up the working environment in *Jupyter* Notebook and prepares all necessary tools for data cleaning and RFM analysis.

Step 2: Load the Raw Dataset

```
df = pd.read_csv('online_retail_II_reduced.csv')
```

```
df
```

Explanation

In this step, the raw transactional dataset is loaded into a Pandas *DataFrame* using the `read_csv()` function. This converts the CSV file into a structured tabular format that can be easily explored and cleaned.

- `pd.read_csv('online_retail_II_reduced.csv')`
Reads the CSV file and stores it in a *DataFrame* called `df`.
- `df`
Displays the dataset in the *Jupyter* Notebook to visually inspect the raw data,

including column names, sample values, missing entries, incorrect data types, and obvious anomalies such as negative quantity or price values.

This step helps in understanding the structure of the raw data and identifying data quality issues before performing any cleaning or transformation.

Step 3: Rename Columns for Consistency

```
#RENAMING COLUMNS TO READABLE FORM
```

```
df.rename(columns={'Customer ID':'Customer_ID'})
```

In this step, the column name Customer ID is renamed to *Customer_ID* to improve consistency and make the column easier to reference in code.

- Column names with spaces can cause inconvenience when selecting columns or writing chained operations.
- Renaming the column ensures a standardized naming convention (using underscores instead of spaces), which improves readability and reduces the chance of syntax errors in later steps.
- The parameter *inplace = True* updates the original *DataFrame* directly without creating a new copy.

This step improves code clarity and prevents potential issues when performing grouping and aggregations for RFM analysis.

Step 4: Check for Missing Values

```
#CHECKING NULL VALUES
```

```
df.isnull().any()
```

Invoice	False
StockCode	False
Description	True
Quantity	False
InvoiceDate	False
Price	False
Customer ID	True
Country	False
dtype:	bool

Explanation

This step is used to identify whether any columns in the dataset contain missing (null) values.

- `df.isnull()`
Creates a boolean DataFrame where each cell is marked as True if the value is missing and False otherwise.
- `.any()`
Checks each column and returns True if at least one missing value exists in that column.

This helps quickly identify problematic columns (such as *Customer_ID* or Description) that contain null values and need to be handled during the data cleaning process.

Detecting missing values early ensures that invalid records do not affect RFM calculations, which require complete customer and transaction information.

	A	B	C	D	E	F	G	H
2221	489597	37446	MINI CAKE STAND WITH HANGING CAKES	2	01-12-2009 14:28	3.43		United Kingdom
2222	489597	37449	CERAMIC CAKE STAND + HANGING CAKES	1	01-12-2009 14:28	21.7		United Kingdom
2223	489597	37462B	PET MUG, BUDGIE	2	01-12-2009 14:28	0.86		United Kingdom
2224	489597	37464	ROBOT MUG IN DISPLAY BOX	1	01-12-2009 14:28	2.57		United Kingdom
2225	489597	37489C	GREEN/BLUE FLOWER DESIGN BIG MUG	1	01-12-2009 14:28	4.3		United Kingdom
2226	489597	37503	TEA TIME CAKE STAND IN GIFT BOX	1	01-12-2009 14:28	21.7		United Kingdom
2227	489597	37508	NEW ENGLAND CERAMIC CAKE SERVER	1	01-12-2009 14:28	5.17		United Kingdom
2228	489597	47471	RAINBOW PEGS PHOTO CLIP STRING	1	01-12-2009 14:28	1.52		United Kingdom
2229	489597	47587A	PINK FAIRY CAKE COASTER	2	01-12-2009 14:28	1.7		United Kingdom
2230	489597	47591C	BLUE FAIRY CAKE CHILD'S APRON	2	01-12-2009 14:28	4.3		United Kingdom
2231	489597	47591D	PINK FAIRY CAKE CHILD'S APRON	1	01-12-2009 14:28	4.3		United Kingdom
2232	489597	47598	NEW ENGLAND EGG WARMER	1	01-12-2009 14:28	1.3		United Kingdom
2233	489597	48129	DOOR MAT TOPIARY	1	01-12-2009 14:28	13.87		United Kingdom
2234	489597	48173C	DOOR MAT BLACK FLOCK	2	01-12-2009 14:28	13.87		United Kingdom
2235	489597	48188	DOOR MAT WELCOME PUPPIES	1	01-12-2009 14:28	13.87		United Kingdom
2236	489597	51014A	FEATHER PEN, HOT PINK	1	01-12-2009 14:28	1.7		United Kingdom
2237	489597	51014C	FEATHER PEN, COAL BLACK	1	01-12-2009 14:28	1.7		United Kingdom
2238	489597	70006	LOVE HEART POCKET WARMER	2	01-12-2009 14:28	3.43		United Kingdom
2239	489597	70007	HI TEC ALPINE HAND WARMER	1	01-12-2009 14:28	3.43		United Kingdom
2240	489597	71143	SILVER BOOK MARK WITH BEADS	2	01-12-2009 14:28	2.57		United Kingdom
2241	489597	72051S	BAG OF SILVER STONES	1	01-12-2009 14:28	0.86		United Kingdom
2242	489597	72598	SET/12 TAPER CANDLES	1	01-12-2009 14:28	1.7		United Kingdom
2243	489597	72741	GRAND CHOCOLATE CANDLE	1	01-12-2009 14:28	8.65		United Kingdom
2244	489597	72799E	IVORY PILLAR CANDLE SILVER FLOCK	1	01-12-2009 14:28	6.04		United Kingdom
2245	489597	72800A	S/4 BLACK DINNER CANDLE SILVER FLOC	1	01-12-2009 14:28	5.17		United Kingdom
2246	489597	72802C	VANILLA SCENT CANDLE JEWELLED BOX	2	01-12-2009 14:28	8.65		United Kingdom
2247	489597	72807A	SET/3 ROSE CANDLE IN JEWELLED BOX	1	01-12-2009 14:28	8.65		United Kingdom
2248	489597	72807B	SET/3 OCEAN SCENT CANDLE JEWEL BOX	1	01-12-2009 14:28	8.65		United Kingdom
2249	489597	72807C	SET/3 VANILLA SCENTED CANDLE IN BOX	1	01-12-2009 14:28	8.65		United Kingdom

Step 5: Remove Records with Missing Customer ID

```
df = df.dropna(subset=['Customer ID'])
df
```

Explanation

In this step, all rows with missing values in the *Customer_ID* column are removed from the dataset.

- RFM analysis is performed at the **customer level**, so each transaction must be associated with a valid customer.
- Records with null Customer IDs cannot be assigned to any customer and would distort Frequency and Monetary calculations.

- `dropna(subset = ['Customer_ID'])` removes only those rows where the `Customer_ID` is missing, while keeping all other valid records intact.

After this step, the dataset contains only transactions that can be reliably linked to individual customers, making it suitable for customer-based analysis such as RFM segmentation.

```
df.isnull().any()
```

```
Invoice      False
StockCode    False
Description   False
Quantity      False
InvoiceDate   False
Price         False
Customer ID   False
Country       False
dtype: bool
```

Step 6: Remove Transactions with Non-Positive Quantity

```
#REMOVING NEGATIVE VALUES
```

```
df = df[df['Quantity']>0]      #Negative Quantity Removed
df
```

Explanation

In this step, all transactions with **zero or negative quantity values** are removed from the dataset.

- Negative quantities usually represent **returns or cancellations**, which do not reflect actual purchases.
- Zero or negative values can distort **Frequency** (number of purchases) and **Monetary** (total spending) calculations.
- Since RFM analysis focuses on positive purchasing behaviour, only records with **Quantity > 0** are retained.

This filtering ensures that the dataset includes only valid sales transactions, improving the accuracy of customer value metrics used in RFM analysis.

Step 7: Remove Transactions with Non-Positive Price

```
df = df[df['Price']>0] #Negative Price Removed
df
```

Explanation

In this step, all records with **zero or negative price values** are removed from the dataset.

- A negative or zero price is invalid for sales transactions and usually indicates **data entry errors, refunds, or system adjustments**.
- Including such values would incorrectly reduce the **Monetary** value of customers and distort revenue calculations.
- By keeping only records with **Price > 0**, the dataset is restricted to valid revenue-generating transactions.

This step ensures that the monetary component of RFM analysis is calculated using only genuine sales values, improving the reliability of customer segmentation results.

Step 8: Convert *InvoiceDate* to Datetime Format

```
#CHANGING FORMATS
```

```
df.loc[:, 'InvoiceDate'] = pd.to_datetime(df['InvoiceDate']) #Converted InvoiceDate to date format.  
df
```

Explanation

In this step, the *InvoiceDate* column is converted from a string/object type to a proper datetime format.

- RFM analysis requires date-based calculations to compute Recency, which measures how recently a customer made a purchase.
- Converting *InvoiceDate* into a datetime format enables time-based operations such as finding the latest transaction date and calculating date differences.

Why .loc is Used (Slicing Issue Explanation)

The `.loc` indexer is used to safely assign values to a column after filtering the *DataFrame* in previous steps (such as removing null customers, negative quantities, and negative prices).

- When a *DataFrame* is filtered, Pandas may create a view instead of a copy, which can lead to the *SettingWithCopyWarning*.
- Using `df.loc[:, 'InvoiceDate'] = ...` ensures that the assignment is applied explicitly to the original *DataFrame* and avoids ambiguity.
- This prevents unintended behaviour where changes might not be saved properly.

This step ensures date values are correctly formatted and safe to use for recency calculations in RFM analysis.

Step 9: Create a Copy of the Cleaned Dataset

```
#AGAIN CHANGING TO ORIGINAL DATAFRAME
```

```
df=df.copy()  
df
```

Explanation

In this step, a **copy of the cleaned *DataFrame*** is created and reassigned back to *df*.

- After multiple filtering and slicing operations, Pandas may return a **view** of the original *DataFrame* instead of a full independent copy.
- This can cause warnings (such as *SettingWithCopyWarning*) or unexpected *behavior* when modifying columns later.
- Using *df.copy()* ensures that all future transformations are applied to a **separate, stable copy** of the data.

This step improves code safety and prevents potential side effects during further feature engineering and RFM calculations.

Step 10: Create Total Transaction Amount (Monetary Value)

```
#CREATING A MONETARY VALUE FOR RFM ANALYSIS
```

```
df['TotalAmount']=df['Quantity'] * df['Price'] #Created monetary value of the product for RFM analysis  
df
```

Explanation

In this step, a new column called ***TotalAmount*** is created by multiplying **Quantity** and **Price** for each transaction.

- The raw dataset does not contain a pre-calculated total sales value per transaction.
- The **Monetary** component of RFM analysis requires the total amount spent by each customer.
- By computing $TotalAmount = Quantity \times Price$, the monetary contribution of each transaction is captured accurately.

This new column forms the basis for calculating the **Monetary value per customer** when aggregating transactions during the RFM feature engineering stage.

Step 11: Final Data Type Validation

```
#FINAL DATA TYPE CHECK
```

```
df.dtypes
```

```
Invoice      object
StockCode    object
Description   object
Quantity      int64
InvoiceDate   object
Price        float64
Customer ID   float64
Country       object
TotalAmount   float64
dtype: object
```

Explanation

In this step, the data types of all columns in the cleaned dataset are reviewed to ensure they are suitable for analysis.

- This confirms that **InvoiceDate** is in datetime format, which is required for calculating **Recency**.
- It verifies that **Quantity**, **Price**, and **TotalAmount** are numeric, which is required for computing **Frequency** and **Monetary** values.
- Ensuring correct data types helps prevent calculation errors and unexpected behaviour during grouping and aggregation.

This final check validates that the dataset has been successfully cleaned and prepared for the RFM feature engineering and customer segmentation steps.

Step 12: Export the Cleaned Dataset

```
#EXPORTING CLEANED DATA
```

```
#df.to_excel("cleaned_data_phase_1.xlsx", index=False) #Imported the file to xlsx format
```

Explanation

In this step, the fully cleaned and pre -processed dataset is exported to an Excel file for future use.

- Saving the cleaned data creates a **reusable, analysis-ready version** of the dataset.

- This allows the cleaned dataset to be shared, reviewed, or used in later phases such as **RFM feature engineering, scoring, and segmentation** without repeating the cleaning process.
- The parameter `index=False` prevents Pandas from writing the *DataFrame* index as an extra column in the Excel file.

This step completes the data basic cleaning phase and creates a stable input file for the RFM advance cleaning phase.

Cleaned Dataset

Invoice (F)	StockCode	Description	Quantity	InvoiceDate (R)	Price	Customer ID	Country	TotalAmount (M)				
489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-01-12 07:45:00	6.95	13085	United Kingdom	83.4	R	Recency		
489434	79323P	PINK CHERRY LIGHTS	12	2009-01-12 07:45:00	6.75	13085	United Kingdom	81	F	Frequency		
489434	79323W	WHITE CHERRY LIGHTS	12	2009-01-12 07:45:00	6.75	13085	United Kingdom	81	M	Monetry		
489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-01-12 07:45:00	2.1	13085	United Kingdom	100.8				
489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-01-12 07:45:00	1.25	13085	United Kingdom	30				
489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-01-12 07:45:00	1.65	13085	United Kingdom	39.6				
489434	21871	SAVE THE PLANET MUG	24	2009-01-12 07:45:00	1.25	13085	United Kingdom	30				
489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-01-12 07:45:00	5.95	13085	United Kingdom	59.5				
489435	22350	CAT BOWL	12	2009-01-12 07:46:00	2.55	13085	United Kingdom	30.6				
489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-01-12 07:46:00	3.75	13085	United Kingdom	45				
489435	22195	HEART MEASURING SPOONS LARGE	24	2009-01-12 07:46:00	1.65	13085	United Kingdom	39.6				
489435	22353	LUNCHBOX WITH CUTLERY FAIRY CAKES	12	2009-01-12 07:46:00	2.55	13085	United Kingdom	30.6				
489436	48173C	DOOR MAT BLACK FLOCK	10	2009-01-12 09:06:00	5.95	13078	United Kingdom	59.5				
489436	21755	LOVE BUILDING BLOCK WORD	18	2009-01-12 09:06:00	5.45	13078	United Kingdom	98.1				
489436	21754	HOME BUILDING BLOCK WORD	3	2009-01-12 09:06:00	5.95	13078	United Kingdom	17.85				
489436	84879	ASSORTED COLOUR BIRD ORNAMENT	16	2009-01-12 09:06:00	1.69	13078	United Kingdom	27.04				
489436	22119	PEACE WOODEN BLOCK LETTERS	3	2009-01-12 09:06:00	6.95	13078	United Kingdom	20.85				
489436	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-01-12 09:06:00	1.45	13078	United Kingdom	17.4				
489436	22296	HEART IVORY TRELLIS LARGE	12	2009-01-12 09:06:00	1.65	13078	United Kingdom	19.8				
489436	22295	HEART FILIGREE DOVE LARGE	12	2009-01-12 09:06:00	1.65	13078	United Kingdom	19.8				
489436	22109	FULL ENGLISH BREAKFAST PLATE	16	2009-01-12 09:06:00	3.39	13078	United Kingdom	54.24				
489436	22107	PIZZA PLATE IN BOX	4	2009-01-12 09:06:00	3.75	13078	United Kingdom	15				
489436	22194	BLACK DINER WALL CLOCK	2	2009-01-12 09:06:00	8.5	13078	United Kingdom	17				
489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	2009-01-12 09:06:00	4.65	13078	United Kingdom	55.8				
489436	82582	AREA PATROLLED METAL SIGN	12	2009-01-12 09:06:00	2.1	13078	United Kingdom	25.2				
489436	21181	PLEASE ONE PERSON METAL SIGN	12	2009-01-12 09:06:00	2.1	13078	United Kingdom	25.2				
489436	21756	BATH BUILDING BLOCK WORD	3	2009-01-12 09:06:00	5.95	13078	United Kingdom	17.85				
489436	21333	CLASSIC WHITE FRAME	6	2009-01-12 09:06:00	2.95	13078	United Kingdom	17.7				

Summary of Data Cleaning Phase

1. The raw transactional dataset was first loaded into *Jupyter* Notebook for exploration and preprocessing.
2. Column names were standardized to improve readability and avoid syntax issues during analysis.
3. Records with missing *Customer_ID* values were removed since RFM analysis requires customer-level aggregation.
4. Transactions with non-positive *Quantity* values were filtered out to exclude returns and cancellations.
5. Transactions with non-positive *Price* values were removed to ensure only valid sales were included.
6. The *InvoiceDate* column was converted into proper datetime format to support recency calculations.

7. A new column, ***TotalAmount***, was created by multiplying Quantity and Price to represent the monetary value of each transaction.
8. Final data type validation was performed to ensure all columns were in suitable formats for analysis.
9. The cleaned dataset was exported as an Excel file to create a reusable, analysis-ready version for the RFM Advance Cleaning Phase.

This process transformed noisy raw transaction data into a clean and structured dataset suitable for accurate RFM advance cleaning phase.